PSF & CLASS — the maths, explained plainly

A short companion to `LEARNOVA_PAPER.md` / `LEARNOVA_PAPER.pdf`, written to talk a reviewer through the formulas in plain language.

The one-sentence version

PSF is a formula that scores how good a slide is *for learning* (0 to 1). **CLASS** is an algorithm that decides how to split content across slides so that score stays as high as possible. They share one idea: the same maths that *judges* a slide also *builds* it.

---

1. Why not just add up a few scores?

The obvious way to score a slide is to add things up:

```
score = 20·(text is a good length) + 20·(bullet count is okay) + 15·(has a title)
+ ...
```

Problem: a slide with **nothing on it** can still score decently — it has a short bullet, a nice title, no clutter. The score rewards *emptiness* by accident. That's the flaw in the old scorer.

Fix: multiply instead of add.

```
PSF(slide) = E ^ α  ×  (1 − L) ^ β  ×  C ^ γ
```

If **any one** of `E`, `(1−L)`, or `C` is close to zero, the *whole product* collapses to near zero — one weak link sinks the whole score, the way a chain is only as strong as its weakest link. This is called a **Cobb–Douglas form** (economists use the same shape to combine "labour" and "capital" into "output" — same logic: both inputs matter, and zero of either gives zero output).

Symbol	Plain English
`E`	**Efficiency** — how much *new, useful* information this slide actually delivers
`L`	**Load** — how hard this slide is to hold in your head while reading it
`C`	**Coherence** — how well the slide is *put together* (has a point, doesn't ramble)
`α, β, γ`	How much each of the three matters, as fractions that add to 1 (e.g. 0.40, 0.42, 0.18) — *tunable knobs, not magic numbers*

Say it out loud to sir as: *"a slide is good only if it says something, doesn't overload the reader, AND hangs together — not if it merely averages okay on those three."*

---

2. `E` — Information efficiency

```
E = ( U / (U + κ) )  ×  (1 − R)
```

- **`U`** = "how much new stuff is on this slide". We count the *rare, meaningful* words the slide introduces that weren't on an earlier slide, and add up how rare each one is (a common word like "system" barely counts; a technical term like "photosystem" counts a lot).
- **`U / (U + κ)`** — this is the important bit. It's a **diminishing-returns curve**: the *first* couple of new ideas raise the score a lot, but the *tenth* new idea on the same slide barely moves it. `κ` (kappa) is just the "how many ideas before it saturates" knob. This is the same curve biologists use for enzyme speed (Michaelis–Menten) — *"more of something helps, but less and less as you add more."*
- **`R`** = redundancy — how much the slide just repeats its own title or itself. `(1 − R)` subtracts that back off.

In one line for sir: *"E rewards a slide for saying one or two genuinely new things clearly — not for cramming in ten things, and not for restating the title."*

---

3. `L` — Cognitive load

```
L = sigmoid( w_e·ê + w_t·t̂ + w_v·v̂ + w_s·ŝ − θ )
```

Four things make a slide *hard to hold in your head at once* (this is **Cognitive Load Theory** from educational psychology — working memory holds roughly 4 things at a time):

Term	What it measures
`ê`	how many ideas interact with each other (not just how many *words*)
`t̂`	how much text there is, relative to a comfortable reading chunk
`v̂`	how visually busy the slide is (table rows, diagram nodes, cards)
`ŝ`	whether a figure is described in the text but shown on a *different* slide (forces the eye to jump)

Each is scaled to sit between `0` and `1` by a reference number and multiplied by a weight `w_*` — the weights just say *how much each factor hurts*, and they're the numbers that get tuned against real student ratings.

`sigmoid(x) = 1 / (1 + e^−x)` — the **S-curve**. Plain meaning: it squashes any number into a clean 0-to-1 range, and it's forgiving in the middle but harsh at the extremes — a slightly-too-long slide isn't punished much, a *wildly* overloaded slide gets punished a lot. `θ` (theta) shifts the curve — how much load is "acceptable" before the penalty kicks in.

In one line for sir: *"L adds up the four things that make a slide feel overwhelming, then runs the total through an S-curve so small overloads are forgiven and big ones are punished hard."*

---

4. `C` — Coherence

Just the **average of four yes/no-ish checks**, each 0 to 1:

1. Does it have a real title *and* a stated takeaway? (signals structure)
2. Do the sentences actually relate to the slide's main topic, or does one wander off? (no "seductive detail")
3. Does the image caption just repeat the bullets, or add something?
4. Is a figure sitting next to the text that talks about it?

No new maths here — just an average of four checks grounded in **Mayer's Cognitive Theory of Multimedia Learning**, a well-known education-research framework.

---

5. Combining slides into a deck score

```
PSF_deck = ( PSF₁ × PSF₂ × ... × PSFₙ ) ^ (1/n)   ×   Flow
```

- The **geometric mean** (product, then n-th root) instead of a plain average — same reasoning as before: *one terrible slide should drag the whole deck down*, not get diluted by nine good ones.
- **`Flow`** rewards *moderate* overlap between consecutive slides — too little overlap feels like a jarring topic jump, too much feels repetitive. It's a **trapezoid function**: flat at 1.0 in a "just right" band, tapering off on both sides.

---

6. CLASS — deciding how to split content across slides

The question: given a list of bullet points for one section, how many go on each slide?

The old way: just cut every N bullets. Simple, but ignores whether those N bullets happen to be a hard combination.

CLASS's way — treat it like a well-known solved problem. This is exactly the same maths word processors use to decide where to break lines of text so a paragraph looks even (**Knuth–Plass line-breaking**, 1981) — except "line" becomes "slide" and "how ugly the line looks" becomes "how much cognitive load the slide has":

```
minimise:  Σ (cost of each slide)  +  λ × (number of slides)
```

- **`cost of a slide`** = its `L` (load) from PSF, plus a penalty for being nearly empty, plus a soft penalty for going over a "comfortable" size.
- **`λ`** (lambda) is a small tax per extra slide — it stops the algorithm from making one bullet per slide just because that minimises load; splitting has a cost too.

How it's actually solved — dynamic programming. Instead of trying every possible way to cut the list (which would take forever for a long section), the algorithm builds the answer up one bullet at a time, always remembering the *cheapest* way to reach that point — the standard trick called **dynamic programming**. It runs in a fraction of a millisecond and is **mathematically guaranteed to find the best possible split**, not just a good one — this is a proven property of this class of algorithm (the same guarantee Knuth & Plass proved for line-breaking).

The guarantee that matters for a teaching tool: because every bullet is assigned to exactly one slide by construction, **content can never be silently dropped** — the algorithm can only choose *where* to cut, never *what* to discard.

In one line for sir: *"It's the same algorithm Word uses to break lines evenly — but instead of making lines look even, it makes slides feel equally easy to digest, and it's provably optimal, not just a good guess."*

---

7. If sir asks… (quick answers)

"Why multiply instead of add the sub-scores?" → So one bad dimension (empty, overloaded, or incoherent) can't be hidden by two good ones. Same logic economists use for output = labour × capital.

"Where do the weights (α, β, γ, w_e, w_t…) come from — did you just guess them?" → They ship with reasonable starting values from the learning-theory literature, and are meant to be *fitted* against real human ratings with a standard curve-fitting method (constrained least squares), the same way you'd calibrate any statistical model — see `scripts/calibrate_psf.py`.

"What guarantees do you actually have, not just claim?" → Two, both provable from how the code is structured: (1) CLASS never drops a bullet — every one is assigned to exactly one slide; (2) the DP finds the *global* minimum-cost split, not just a locally good one, because the cost of a slide only depends on what's on it, which is exactly the condition needed for that proof to hold.

"Is this machine learning?" → No trained model, no neural network. It's a formula with a handful of tunable numbers (calibrated the way you'd calibrate any scientific model) and an optimal-search algorithm — both fully deterministic and inspectable.

"What's the actual research contribution?" → Turning two education-research theories (Cognitive Load Theory, Mayer's Multimedia Learning) into one *computable* formula, and then reusing that exact formula as the cost function of an optimal slide-splitting algorithm — so the same maths both judges a slide and builds it.

---

8. A tiny worked example

A slide with 2 new technical terms, 40 words, no figure:

```
U ≈ 3.4   (two rare terms, idf-weighted)
E = 3.4 / (3.4 + 6.0) = 0.36        →  a modest, single-idea slide
```

```
ê, t̂ come out small (short, simple)  →  L ≈ 0.18   (low load — good)

C ≈ 0.8   (has a title + takeaway, on-topic)

PSF = 0.36^0.40 × (1 − 0.18)^0.42 × 0.8^0.18
    = 0.64 × 0.92 × 0.96  ≈ 0.57  →  57 / 100
```

A "wall of text" slide (12 bullets, 4 competing ideas, no takeaway) instead gets `E≈0.6, L≈0.75, C≈0.3`, and:

```
PSF = 0.6^0.40 × 0.25^0.42 × 0.3^0.18 ≈ 0.82 × 0.55 × 0.81 ≈ 0.36  →  36/100
```

Even though it "says more" (`E` is higher), the **load and incoherence pull it down hard** — exactly the multiplicative effect §1 describes.